

OBJECT-ORIENTED PROGRAMMING

WEEK 1 LAB EVIDENCE

Thinking in Objects: From Everyday Things to Java Classes

Student Name	KHADIJA ABID
Seat Number	B25110006065
Section	B
Date	25 July 2026
Lab Number	01

COMPLETED EVIDENCE

- Five everyday objects with relevant attributes and behaviours.
- A UML diagram for the selected Laptop object.
- An enhanced Car UML and matching multi-file Java program.
- A Calculator UML and matching Java program with normal and edge cases.
- A stack and heap diagram explaining instantiation, references and null.
- A short reflection, assignment plan and pre-submission check.

All Java source files were compiled and run successfully.

The selected objects are common items that I use as a student. Only attributes and behaviours useful to a simple software model are included.

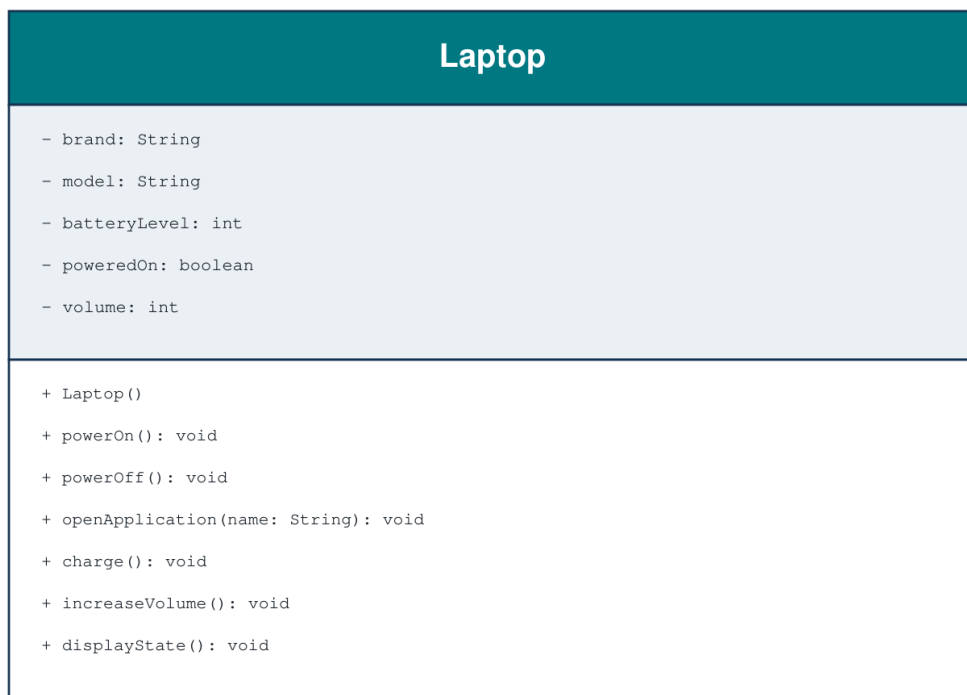
Object	Relevant attributes	Behaviours
1. Laptop	brand, model, RAM, storage, batteryLevel, poweredOn	powerOn(), powerOff(), runProgram(), charge(), connectWiFi()
2. Mobile Phone	brand, model, batteryLevel, storage, volume	call(), sendMessage(), takePhoto(), openApp(), charge()
3. Headphones	brand, volume, batteryLevel, connected, playing	connect(), play(), pause(), increaseVolume(), charge()
4. Backpack	color, capacity, compartments, items, zipOpen	open(), close(), addItem(), removeItem(), carry()
5. Bicycle	brand, color, speed, gear, brakeApplied	pedal(), brake(), changeGear(), turnLeft(), turnRight()

Selected object for an additional UML diagram: Laptop

The Laptop class models only the details required for basic power, battery, application and volume behaviour.

UML CLASS DIAGRAM - LAPTOP

Student-selected everyday object.

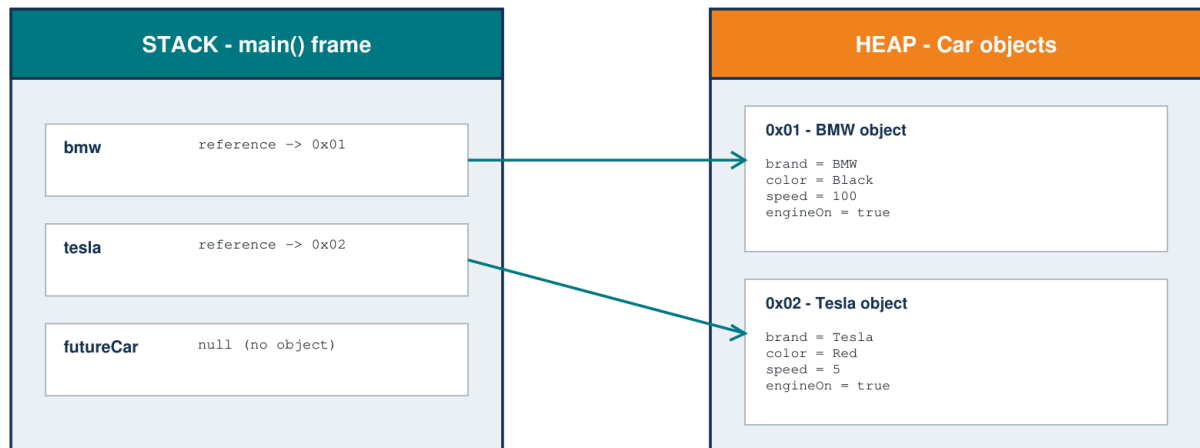


- private + public

Abstraction means selecting the useful features of the real object and leaving out details that the program does not need.

STACK AND HEAP FOR THE CAR OBJECTS

Reference variables are stored in the main method stack frame. The objects created with new are stored in heap memory.



COMMENTED CODE REPRESENTATION

```
// Reference variable + instantiation of a new object
Car bmw = new Car();

// A second reference points to a different object
Car tesla = new Car();

// Null construction: this reference does not point to an object
Car futureCar = null;
```

BEGINNER EXPLANATION

- bmw and tesla are reference variables in the stack frame of main().
- new Car() creates a separate object in heap memory and runs the constructor.
- The reference stored in bmw points to the BMW object; the reference stored in tesla points to the Tesla object.
- futureCar contains null, so it does not point to any object.
- The two Car objects have independent field values even though they were created from the same class.

SHORT REFLECTION

This lab helped me understand that a class is a design, while an object is a separate instance created from that design. The BMW and Tesla objects use the same Car class, but changes made to one object do not change the other. I also understood why private fields are useful: the methods can check rules before changing the state. Testing invalid movement and division by zero showed me that a program should handle incorrect operations clearly instead of producing a wrong result.

ASSIGNMENT PLAN

- Review the requirements and update both UML diagrams before coding.
- Add Car attributes, movement methods, state rules and the turbo feature.
- Add Calculator operations and tests for normal and edge cases.
- Compile and run both projects, then save the actual output.
- Compare every UML field and method with the Java source before submission.

PRE-SUBMISSION SELF-CHECK

- [X] I can explain every field and method in my UML.
- [X] I understand the objects created in Main.
- [X] I can explain where this is used and why.
- [X] I can explain what the dot operator selects.
- [X] I tested state changes and invalid or edge cases.
- [X] I removed unused code and checked compilation.

Result: Car.java, Calculator.java and both Main.java files compile without errors. The complete run is recorded in SampleOutput.txt.